

The essential software requirement

Waarom hebben we software requirements nodig?

Duidelijke doelstellingen: Requirements helpen bij het vaststellen van duidelijke doelstellingen voor het project. Ze definiëren wat het eindproduct of de dienst moet bereiken en stellen een gemeenschappelijk begrip vast tussen alle belanghebbenden. Dit voorkomt misverstanden en zorgt ervoor dat het team een gezamenlijk doel nastreeft.

Stakeholder verwachtingen beheren: Verschillende belanghebbenden, zoals klanten, gebruikers, ontwikkelaars en testers, hebben vaak verschillende verwachtingen van een project. Requirements dienen als een formele en gestructureerde manier om deze verwachtingen vast te leggen en te beheren. Dit minimaliseert het risico op conflicten en helpt bij het creëren van een product dat aan de verwachtingen van alle partijen voldoet.

Controle over scope en veranderingen: Requirements stellen de scope van het project vast, wat betekent dat ze bepalen wat wel en niet inbegrepen is in het project. Dit helpt bij het voorkomen van scope-creep, waarbij ongeplande functies of wijzigingen worden toegevoegd. Goed gedefinieerde requirements maken het ook gemakkelijker om wijzigingsverzoeken te beoordelen en te beheren, waardoor het project op koers blijft.

Richtlijnen voor ontwerp en ontwikkeling: Requirements fungeren als richtlijnen voor het ontwerp en de ontwikkeling van het project. Ze bieden het ontwikkelingsteam een blauwdruk van wat er moet worden bereikt. Hierdoor kunnen ontwikkelaars gestructureerd werken en wordt de kans op fouten verminderd. Het helpt ook bij het identificeren van eventuele hiaten of onduidelikheden in het begin van het project, waardoor de efficiëntie wordt verhoogd.

Testbaarheid en kwaliteitsborging: Goed gedefinieerde requirements vergemakkelijken het testproces. Test cases kunnen worden afgeleid van de requirements, wat zorgt voor een gestructureerde en systematische aanpak van kwaliteitsborging. Dit verhoogt de kans op het ontdekken en corrigeren van fouten in een vroeg stadium van het project, waardoor de kosten en moeite voor reparaties later worden verminderd.

Wie heeft requirements nodig?

Stakeholders = Iedereen die belang heeft bij het project.

Communicatie is belangrijk

To capture the need or problem completely and unambiguously without resorting to specialist jargon, thus understandable to our customer

Waarom gaan projecten fout

Project Success Factors	% of Responses	Project Impaired Factors	% of Responses	Project Challenged Factors	% of Responses
1. User Involvement	15.9%	1. Incomplete Requirements	13.1%	1. Lack of User Input	12.8%
2. Executive Management Support	13.9%	2. Lack of User Involvement	12.4%	2. Incomplete Requirements & Specifications	12.3%
3. Clear Statement of Requirements	13.0%	3. Lack of Resources	10.6%	3. Changing Requirements & Specifications	11.8%
4. Proper Planning	9.6%	4. Unrealistic Expectations	9.9%	4. Lack of Executive Support	7.5%
5. Realistic Expectations	8.2%	5. Lack of Executive Support	9.3%	5. Technology Incompetence	7.0%
6. Smaller Project Milestones	7.7%	6. Changing Requirements & Specifications	8.7%	6. Lack of Resources	6.4%
7. Competent Staff	7.2%	7. Lack of Planning	8.1%	7. Unrealistic Expectations	5.9%
8. Ownership	5.3%	8. Didn't Need It Any Longer	7.5%	8. Unclear Objectives	5.3%
9. Clear Vision & Objectives	2.9%	9. Lack of IT Management	6.2%	9. Unrealistic Time Frames	4.3%
10. Hard-Working, Focused Staff	2.4%	10. Technology Literacy	4.3%	10. New Technology	3.7%
Other	13.9%	Other	9.9%	Other	23.0%

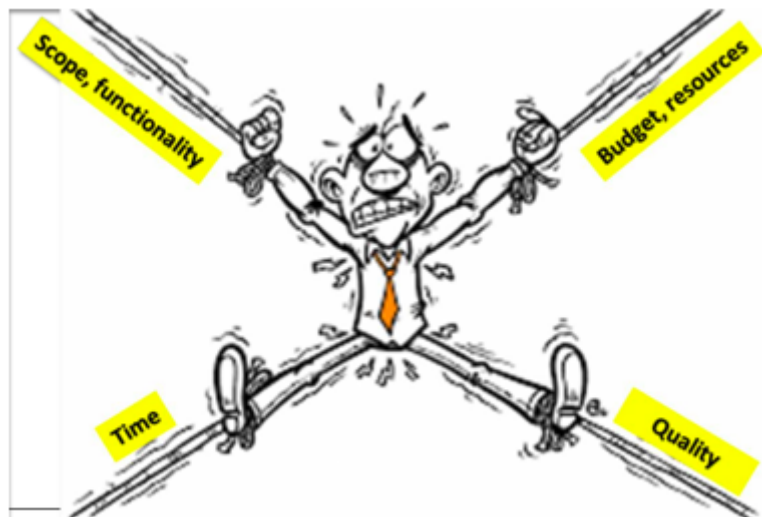
Als een project fout gaat ligt dat vaak aan de requirements, ofwel veranderen ze de hele tijd ofwel zijn ze niet afgewerkt.

De meest genoemde oorzaak dat projecten falen is dat de requirements veranderen.

Requirements management wordt gezien als het grootste probleem in softwareontwikkeling.

Duivelsvierkant

- **Scope:** De functionaliteiten en kenmerken van het project. Het kan gaan om het toevoegen van nieuwe functies of het wijzigen van bestaande functies.
- **Tijd:** De periode die beschikbaar is om het project te voltooien. Het omvat deadlines en tijdlijnen voor oplevering.
- **Kosten:** Het budget dat beschikbaar is voor het project. Dit omvat zowel financiële als middelen budgetten.
- **Kwaliteit:** Dit wordt soms ook genoemd als het vierde element. Het vertegenwoordigt de kwaliteit van het eindproduct of de dienst.



Wat zijn requirements?

Requirements = specificaties van wat geïmplementeerd moet worden. Ze zijn beschrijvingen van hoe het systeem zich moet gedragen, of van een eigenschap of kenmerk

van het systeem. Ze kunnen een beperking vormen voor het ontwikkelingsproces van het systeem.

- Contract tussen de stakeholder en de project manager.
- Een toestand of mogelijkheid die een gebruiker nodig heeft om een probleem op te lossen of een doel te bereiken.

Volgens IEEE(*) is een requirement:

- Een toestand of mogelijkheid die een gebruiker nodig heeft om een probleem op te lossen of een doel te bereiken.
- Een voorwaarde of mogelijkheid waaraan een systeem of systeemcomponent moet voldoen of waaraan een systeem of systeemcomponent moet voldoen om te voldoen aan een contract, standaard, specificatie of andere formeel opgelegde documenten.
- Een gedocumenteerde weergave van een toestand of vermogen zoals hierboven.

Levels en types van requirements

Functionele requirements = definieert een functie die moet worden aangeboden door het te creëren systeem of een van de componenten ervan.

Non-functionele requirements:

- **Quality requirements** = definieert een kwalitatieve eigenschap die het te creëren systeem of een van zijn functies te bieden heeft.
- **Constraint** = Een eis die de oplossingsruimte beperkt tot meer dan nodig is om aan de gegeven functionele eisen en kwaliteitseisen te voldoen.

Business requirement = De functionele of prestatiegerelateerde eisen die zijn vastgesteld door de organisatie om haar doelstellingen te bereiken.

- Sponsor point of view.
- Scope of the project.
- Business objectives.

User requirement = De specifieke behoeften en verwachtingen van de eindgebruiker met betrekking tot een systeem of product.

- User point of view.
- User goals.
- User input and output.
- **Business rule** = Een specifieke instructie of beleid dat de bedrijfsprocessen begeleidt en beïnvloedt, vaak geformaliseerd en geautomatiseerd in een informatiesysteem.
- **Quality attribute** = Een meetbare eigenschap van een systeem die de mate van acceptatie bepaalt, zoals prestaties, betrouwbaarheid, beveiliging en bruikbaarheid,
- Problem domain.

System requirement = Een specificatie van wat het systeem moet doen en hoe het moet presteren, vaak verdeeld in functionele en niet-functionele vereisten.

- **Functional requirement** = Een specificatie van een systeem functie of -functies, beschreven in termen van functionaliteit die het systeem moet bieden.
 - What the system does.
- **Nonfunctional requirement** = Een vereiste die de kenmerken beschrijft die niet direct verband houden met de functionaliteit van het systeem, zoals prestaties, betrouwbaarheid en gebruiksvriendelijkheid.
 - How well the system does it.
- **Constraint** = Een beperkende factor die de ontwikkeling of implementatie van een systeem beïnvloedt.
- **Feature** = Een kenmerk of mogelijkheid van een systeem dat waarde toevoegt voor de gebruiker.
- **External interface requirement** = De specificatie van de interactie tussen een systeem en zijn externe omgeving, inclusief andere systemen, hardware en gebruikers.
- Solution domain.

Best practices: international standards – ISO

ISO 25010 - product kwaliteit

- **Functionality** = Dit kenmerk vertegenwoordigt de mate waarin een product of systeem functies biedt die voldoen aan gestelde en impliciete behoeften bij gebruik onder gespecificeerde omstandigheden.
- **Performance Efficiency** = Dit kenmerk vertegenwoordigt de prestatie in verhouding tot de hoeveelheid middelen die onder de aangegeven omstandigheden worden gebruikt.
- **Compatibility** = Mate waarin een product, systeem of component informatie kan uitwisselen met andere producten, systemen of componenten, en/of de vereiste functies kan uitvoeren terwijl hij dezelfde hardware- of software omgeving deelt.
- **Usability** = Mate waarin een product of systeem door gespecificeerde gebruikers kan worden gebruikt om gespecificeerde doelen met effectiviteit, efficiëntie en tevredenheid te bereiken in een gespecificeerde gebruikscontext.
- **Reliability** = Mate waarin een systeem, product of component gespecificeerde functies uitvoert onder gespecificeerde omstandigheden gedurende een gespecificeerde periode.
- **Security** = Mate waarin een product of systeem informatie en gegevens beschermt, zodat personen of andere producten of systemen de mate van gegevenstoegang hebben die past bij hun soorten en autorisatieniveaus.
- **Maintainability** = Dit kenmerk vertegenwoordigt de mate van effectiviteit en efficiëntie waarmee een product of systeem kan worden aangepast om het te verbeteren, te corrigeren of aan te passen aan veranderingen in de omgeving en in de vereisten.
- **Portability** = Mate van effectiviteit en efficiëntie waarmee een systeem, product of component kan worden overgedragen van de ene hardware, software of andere operationele of gebruiksomgeving naar de andere.



SMART requirements

SMART-principe = management voor het eenvoudig opstellen en controleren van doelstellingen.

- **Specifiek** - Is de doelstelling eenduidig?
- **Meetbaar** - Onder welke voorwaarden of vorm is het doel bereikt?
- **Acceptabel** - Zijn deze doelen acceptabel voor de doelgroep en/of management?
- **Realistisch** - Is het doel haalbaar?
- **Tijdgebonden** - Wanneer moet het doel bereikt zijn?

JIRA - Epics, features, user stories, ...

Epic = groot, overkoepelend thema dat uit meerdere functionaliteiten bestaat. Het is te groot om in één ontwikkelingscyclus te worden voltooid en moet worden opgesplitst in kleinere eenheden, zoals features of user stories.

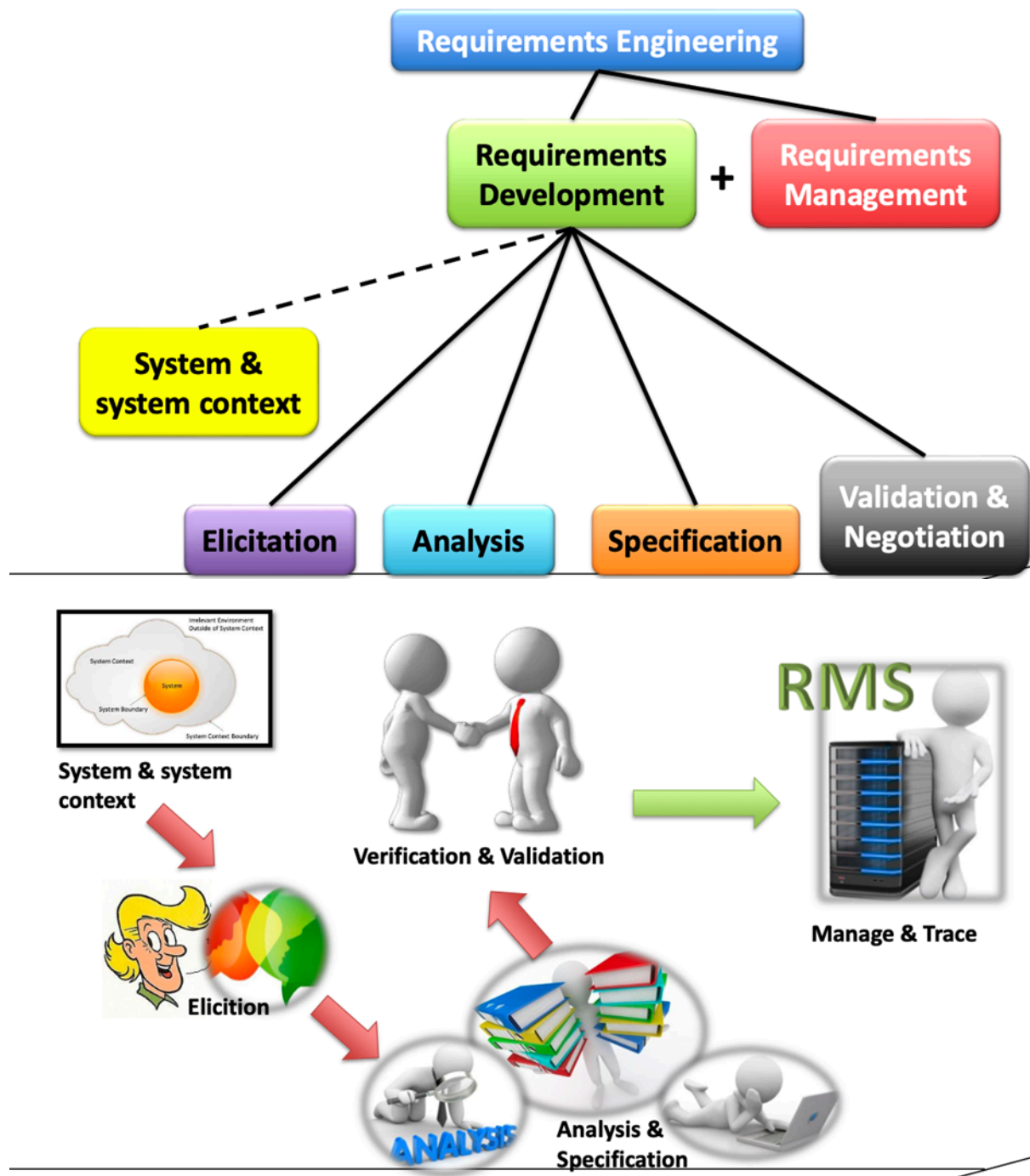
Feature = afgebakend stuk functionaliteit dat samen met andere features een epic vervult. Het kan meerdere user stories bevatten.



User story = korte beschrijving van een functie, verteld vanuit het perspectief van de eindgebruiker. Het is een kleine, zelfstandige eenheid die binnen één ontwikkelingscyclus kan worden voltooid.

- **Functionele user story** = specifieke functionaliteit of kenmerk die waarde toevoegt aan de belanghebbenden. Richten zich op specifieke functies die door de gebruiker kunnen worden waargenomen.
 - "Als gebruiker wil ik de mogelijkheid hebben om in te loggen met mijn Google-account."
- **Niet functionele user story** = beschrijven kwaliteitskenmerken of niet functionele vereisten van het systeem, zoals prestaties, betrouwbaarheid, beveiliging, etc.. Richten zich op kwaliteitsaspecten en prestatiekenmerken die niet direct gerelateerd zijn aan specifieke functies, maar eerder aan het algemene gedrag van het systeem.
 - "Als systeembeheerder wil ik dat het systeem in staat is om 99,9% beschikbaarheid te behouden."

Requirements development and management



Requirement engineering = Een systematische en gedisciplineerde aanpak van de specificatie(ontwikkeling) en het beheer van eisen met de volgende doelstellingen:

- Het kennen van de relevante vereisten, het bereiken van een consensus onder de belanghebbenden over deze vereisten, het documenteren ervan volgens de gegeven normen, en het systematisch beheer ervan.
- De wensen en behoeften van de belanghebbenden begrijpen en documenteren.

- Het specificeren en beheren van vereisten om het risico te minimaliseren dat een systeem wordt opgeleverd dat niet voldoet aan de wensen en behoeften van de belanghebbenden.

Requirements management

- Het proces van het beheren van bestaande vereisten en vereisten gerelateerde artefacten.
- Omvat vooral het opslaan, wijzigen en traceren van vereisten.

Redenen voor onvoldoende requirements engineering:

- Verkeerde veronderstellingen van belanghebbenden: “veel is vanzelfsprekend”.
- Communicatie problemen:
 - Ervaring.
 - Kennis.
- Project Druk van de klant.

Er is geen een oplossing hiervoor:

- Zakelijk: soort systeem, toepasselijke normen, belang van kwaliteitsvereisten, (RAMS: Reliability, Availability, Maintainability, Safety).
- Project:
 - Uitbesteding, omvang, aantal en beschikbaarheid van belanghebbenden, betrokkenheid van de klant.
 - Ontwikkelingsproces: waterval, V-Model, iteratief, incrementeel, agile.
- Menselijke factoren: persoonlijke profielen, politieke verschillen, culturele verschillen.

Informele beschrijvingen (ongestructureerd, geen regels van toepassing)

- Subjectief en daardoor vatbaar voor misverstanden.
- Geen universele taal.

Formele beschrijvingen (gedefinieerde regels)

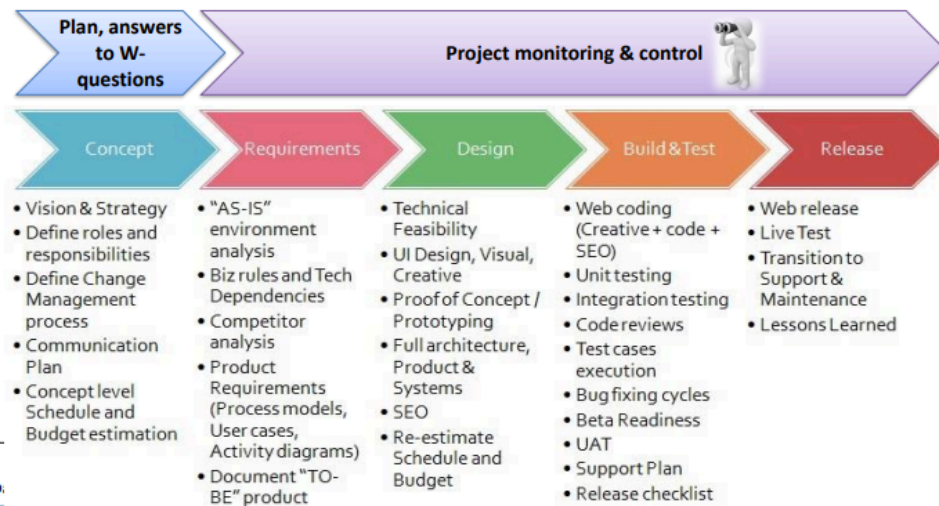
- Geen misverstanden, herbruikbaar.
- De beschrijving is geschikt voor de zaak waar het om gaat.

20 - 80 Rule in communication

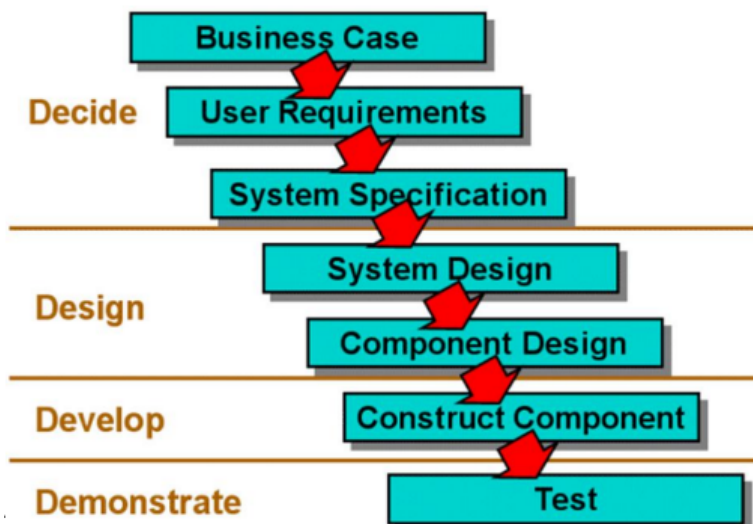
- 20% WHAT is said (words).
- 30 % on HOW it is said (tone used).
- 50% by the EXPRESSION (face, attitude).

Brief history of requirements methods & modeling

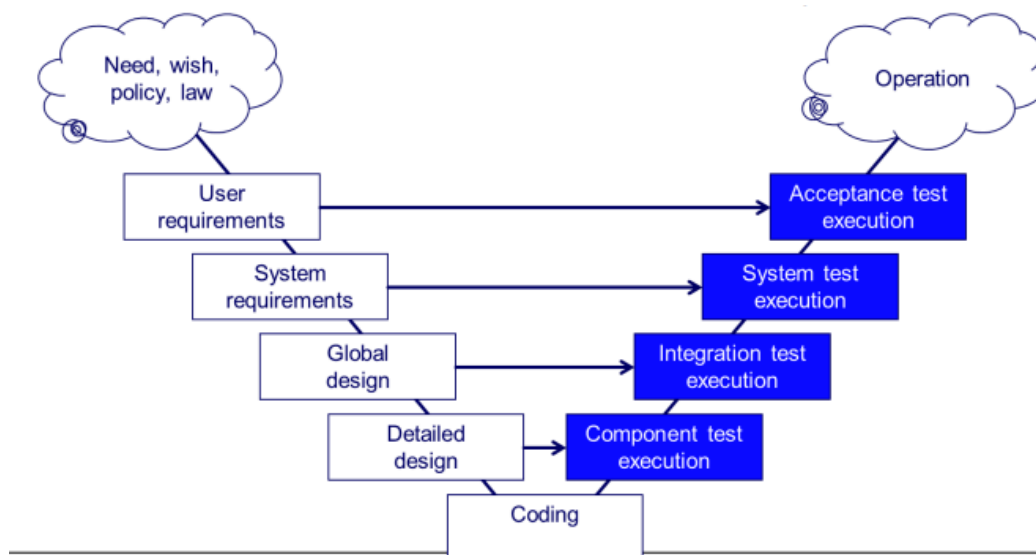
Waar valt software analyse onder?



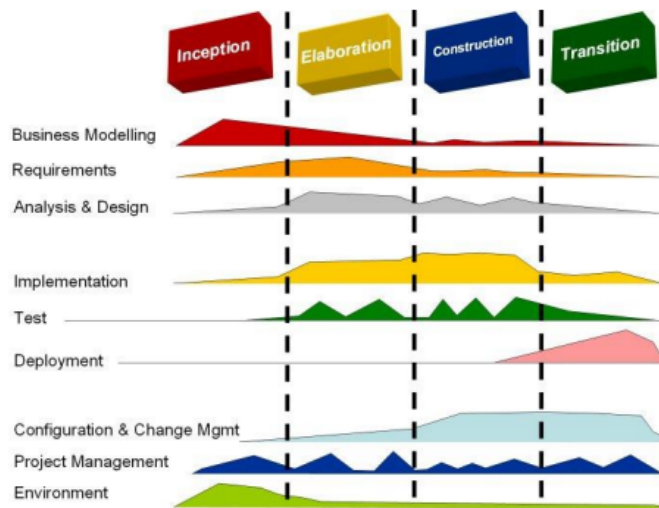
Waterfall model



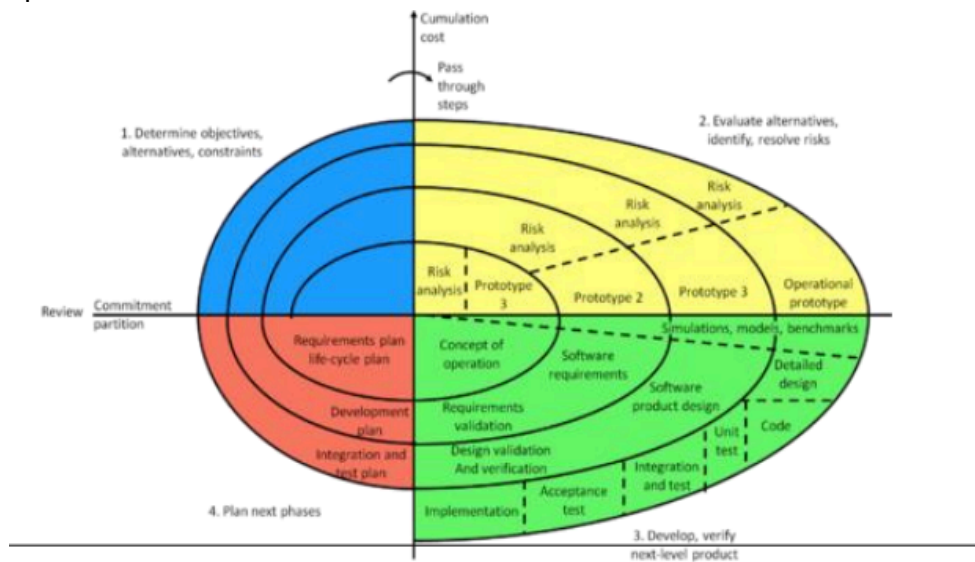
V-model



RUP



Spiral



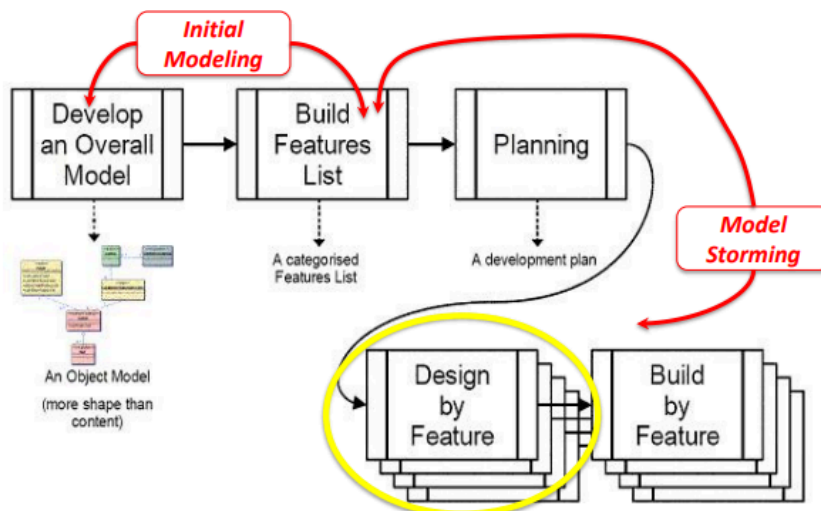
RAD



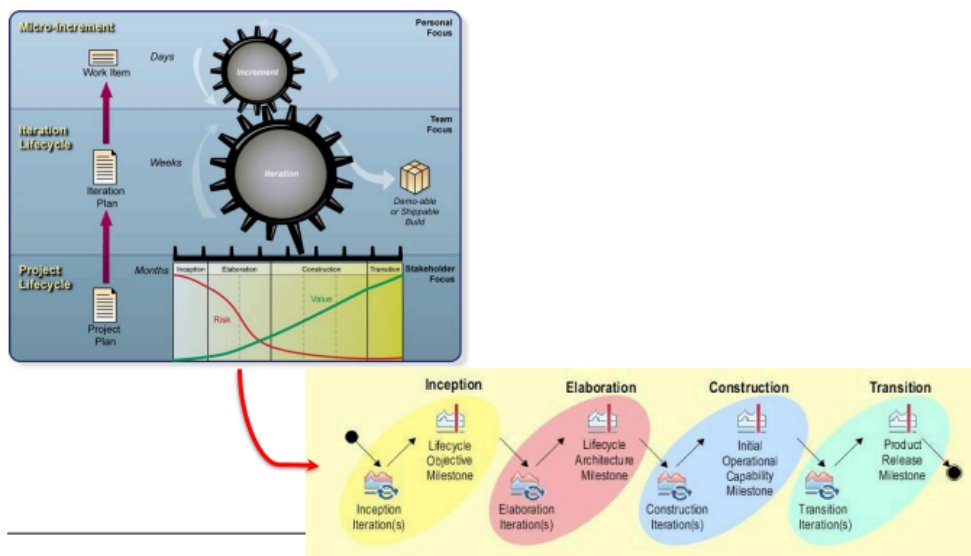
DSDM



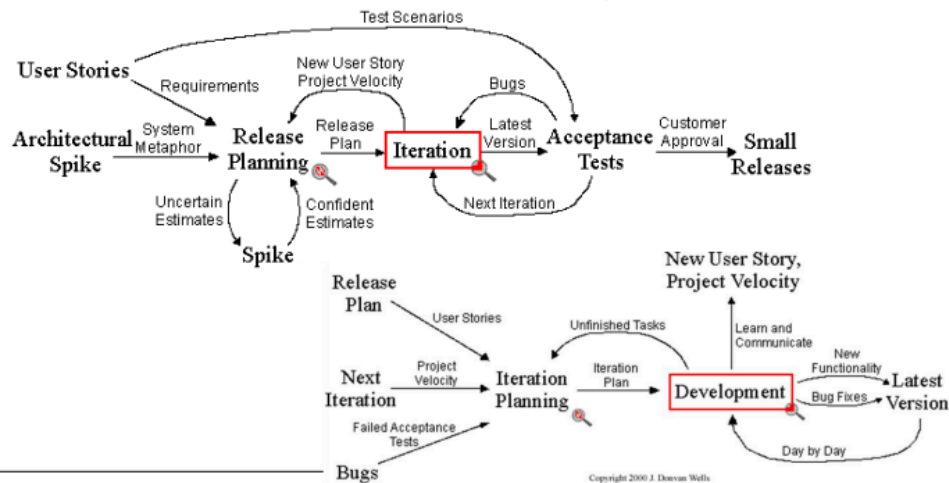
FDD



OpenUP



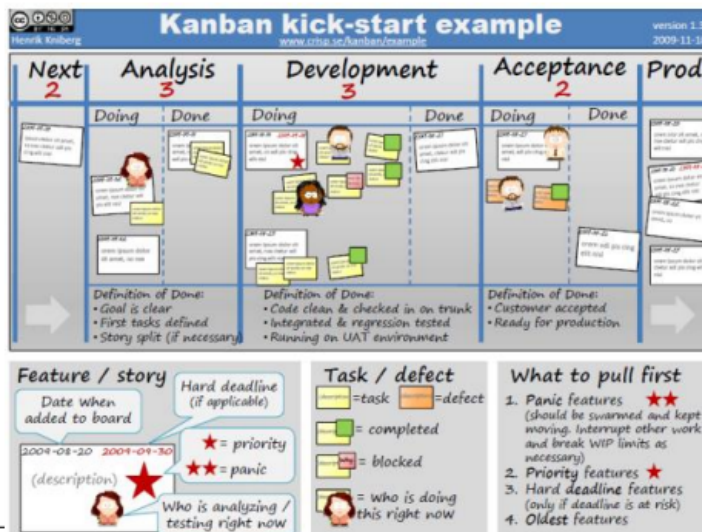
XP



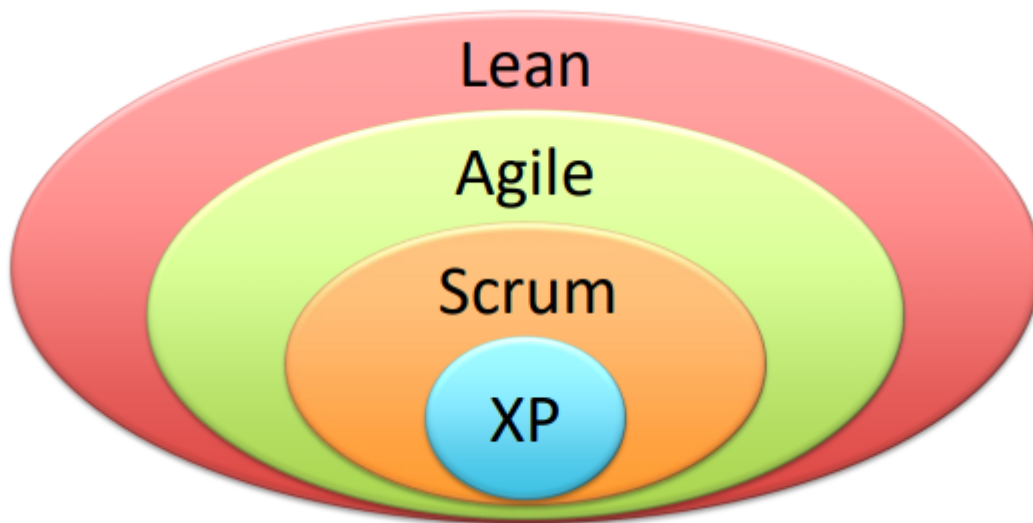
SCRUM



KANBAN

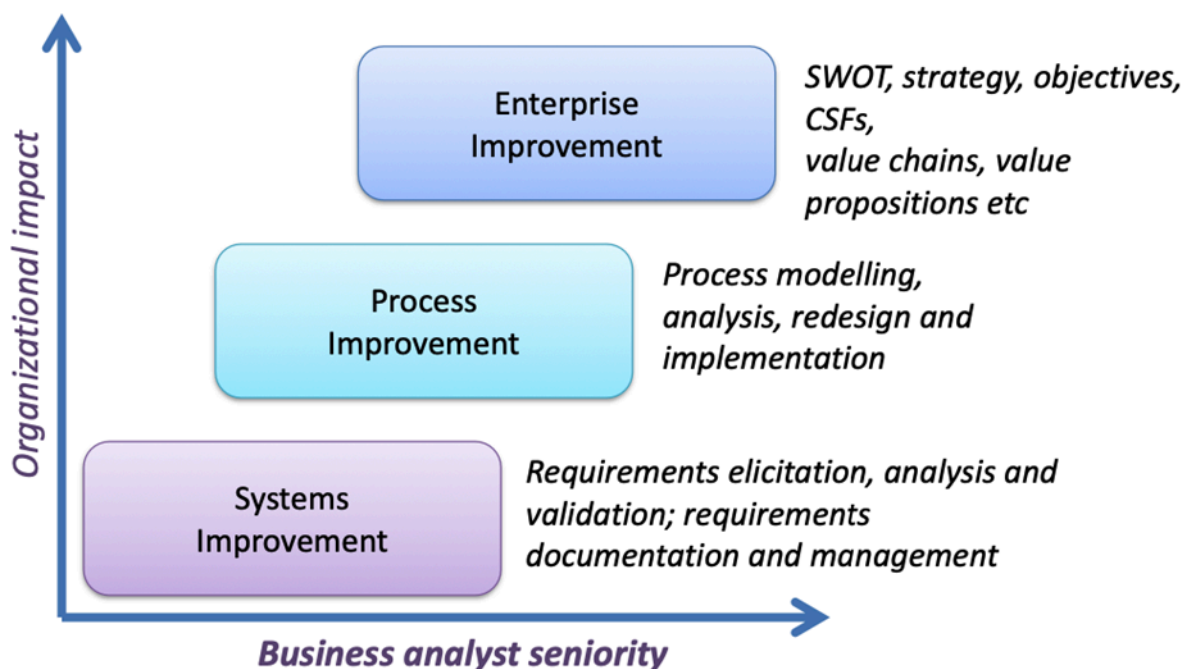


Lean



Role of the analyst

Business analyst = professional die bedrijfsprocessen analyseert, bedrijfsbehoeften identificeert en technologische oplossingen voorstelt om de efficiëntie en effectiviteit van een organisatie te verbeteren.



Activiteiten van een business analyst

- Analyseren en synthetiseren van informatie verstrekt door een groot aantal mensen.
- Verantwoordelijk voor het ontlokken van de werkelijke behoeften van belanghebbenden, niet alleen hun uitgesproken wensen.
- Faciliteren van communicatie tussen organisatie-eenheden.

- Afstemmen van de behoeften van bedrijfseenheden met informatietechnologie.
- Kan fungeren als een "vertaler" tussen bedrijfseenheden.

IIBA Onderliggende competenties

- **Analytisch denken en probleemoplossend vermogen** - probleemoplossing, creatief denken, systeemdenken, leren en besluitvorming
- **Gedragsskenmerken** - ethiek, persoonlijke organisatie en betrouwbaarheid
- **Bedrijfskennis** - zakelijke principes en praktijken, kennis van de industrie, organisatiekennis en oplossings kennis
- **Communicatieve vaardigheden** - mondelinge communicatie, schriftelijke communicatie en onderwijs
- **Interactievaardigheden** - faciliteren en onderhandelen, leiderschap en beïnvloeding, en teamwork
- **Softwaretoepassingen** - algemene toepassingen en gespecialiseerde toepassingen

Requirements engineer

- Analytical thinking & problem solving
 - Creative thinking.
 - Decision making.
 - Learning
 - Problem solving.
 - Systems thinking.
- Behavioral characteristics.
 - Ethics.
 - Personal organization.
 - Trustworthiness.
- Business knowledge.
 - Business principles and practices.
 - Industry knowledge.
 - Organization knowledge.
 - Solution knowledge.
- Communication skills.
 - Oral communication.
 - Teaching.
 - Written communication.
 - Listening.
- Interaction skills.
 - Interaction skills.
 - Leadership and influencing.
 - Teamwork.
- Software applications.
 - General purpose applications.
 - Specialized applications.

Requirements knowledge

- Requirements principles
- Techniques & methods
- Tools

Domain knowledge

- Business process
- User characteristics

IT knowledge

- Software engineering
- Test engineering
- Life cycle models
- Architecture

Soft skills

- Communication
- Analytical
- Change management

Soft skills zijn belangrijk.

Subdisciplines of requirements engineering

Requirements engineering = discipline in software engineering die zich richt op het identificeren, vaststellen, documenteren, valideren en beheren van de vereisten voor een systeem.

- **Requirements development:**
 - **Elicitation** (verzamelen): verzamelen van vereisten van belanghebbenden, zoals klanten, gebruikers en andere belanghebbenden. Het doel is om een volledig beeld te krijgen van de functionaliteiten en eigenschappen die het systeem moet hebben.
 - **Analysis** (analyse): analyseren van de verzamelde vereisten om te begrijpen wat de klant of gebruiker nodig heeft, en om conflicten of inconsistenties te identificeren. Hier wordt gekeken naar de haalbaarheid, relevantie en prioriteit van de verzamelde vereisten.
 - **Specification** (specificatie): formuleren en documenteren van de vereisten op een gestructureerde manier, zodat ze begrijpelijk zijn voor alle belanghebbenden. Dit helpt bij het creëren van een duidelijke en begrijpelijke basis voor verdere ontwikkeling en evaluatie.
 - **Validation** (valideren): controleren en valideren van de gespecificeerde vereisten om er zeker van te zijn dat ze voldoen aan de verwachtingen van de belanghebbenden.
- **Requirements management:**
 - **Tracking** (bijhouden): proces van het volgen van wijzigingen in de vereisten gedurende de ontwikkelingsfase. Hierdoor kunnen projectmanagers en teams de voortgang van de vereisten bijhouden en zorgen voor consistente communicatie met belanghebbenden.
 - **Managing** (beheren): beheren van wijzigingen in de vereisten, inclusief het vaststellen van prioriteiten en besluitvorming over welke wijzigingen moeten worden doorgevoerd. Helpt bij het behouden van de controle over de evoluerende vereisten gedurende de ontwikkelingscyclus.
 - **Controlling** (controleren): implementeren van processen en procedures om de vereisten consistent en beheersbaar te houden. Zorgt ervoor dat het vereistenproces ordelijk blijft en dat wijzigingen op een gecontroleerde manier worden doorgevoerd.
 - **Tracing** (traceren): vastleggen van de relaties tussen vereisten, zodat wijzigingen in één vereiste kunnen worden getraceerd naar de impact op andere vereisten. Het vergemakkelijkt impactanalyse en helpt bij het begrijpen van de gevolgen van wijzigingen gedurende de ontwikkelingsfase.

System and system context

Lanceren van de requirements fase

- Om consensus te bereiken onder de belangrijkste belanghebbenden.
- Om ervoor te zorgen dat je voldoende weet om met het elicitation van requirements te beginnen.

- Om ervoor te zorgen dat het project haalbaar is.
- Om de scope van het uit te voeren werk te definiëren.

A successful project needs precise goals and clear-cut constraints!

Stakeholders = menselijke samenleving die enige invloed op het succes of falen van het project. Iemand die iets wint of verliest als gevolg van het project.

Goals = definieer succescriteria voor het project. Wordt gebruikt om het project te sturen en om het projectteam te helpen keuzes te maken over waar ze hun inspanningen op moeten richten. Hoe weet je of het project wel of niet succesvol is?



Scope = bepaalt de grenzen van het onderzoek en de grenzen van het product dat door het project moet worden gebouwd.

IEEE 830 – SRS template

1. **Introduction** (Purpose. Document conventions. Project Scope. References)
 - 1.1 Purpose: business problem, goals of the project. *Get stakeholders' commitment on this.*
 - Goals of the project - PAM:
 - What is the purpose?
 - What is the business advantage?
 - How will you measure the advantage?
 - 1.2 Product scope → Vision & Scope document: a person or organization that has a (direct or indirect) influence on a system's requirements. *Forgotten stakeholders means forgotten requirements.*
 - 1.3 Glossary: naming conventions and definitions.
 - 1.4 References.
 - 1.5 Overview.
2. **Overall Description** (Product perspective. User classes and characteristics. Operating environment. Design and implementation constraints. Assumptions and dependencies)
 - 2.1 Product perspective: users of the product, make the scope as clear as possible. *Both system boundary and context boundary can shift over time.*
 - 2.2 User classes and characteristics.
 - 2.3 Operating environment.
 - 2.4 Design and implementation constraints.
 - 2.5 User documentation.
 - 2.6 Assumptions and dependencies.
3. **System Features** (System feature x1. Description. Functional requirements. System feature x2,...)
4. **Data Requirements** (Logical data model. Data dictionary. Reports. Data acquisition, integrity, retention, and disposal)
5. **External Interface Requirements** (User interfaces. Software interfaces. Hardware interfaces. Communications interfaces)

6. **Quality Attributes** (Usability. Performance. Security. Safety. Others)
7. **Internationalization and Localization Requirements**
8. **Other Requirements**

Appendix A: Glossary

Appendix B: Analysis Models

System context

- Bron van vereisten voor een systeem.
- Bron = "aspecten die de definitie van de vereisten hebben geïnitieerd of beïnvloedt."
- Potentiële aspecten:
 - Personen: stakeholders of groepen belanghebbenden.
 - Systemen: technische systemen, software en hardware.
 - Processen: technisch, fysiek of bedrijfsprocessen.
 - Gebeurtenissen: technisch of fysiek.
 - Documenten: wetten, normen of systeemdocumentatie.

System boundary

- Welke aspecten moeten door het systeem worden afgedekt?
- Welke aspecten moeten in de omgeving van het systeem blijven?

System context and boundaries

- Context diagrammen:
 - Data flow diagram level zero.
 - Bronnen in de omgeving worden gemodelleerd (bijvoorbeeld de oorsprong of bestemming van informatiestromen tussen het systeem en de omgeving).
- Business use case diagrammen:
 - Actoren (personen of andere systemen) in de omgeving met hun relatie tot (de use cases van) het systeem worden gemodelleerd.
- Domein modellen.

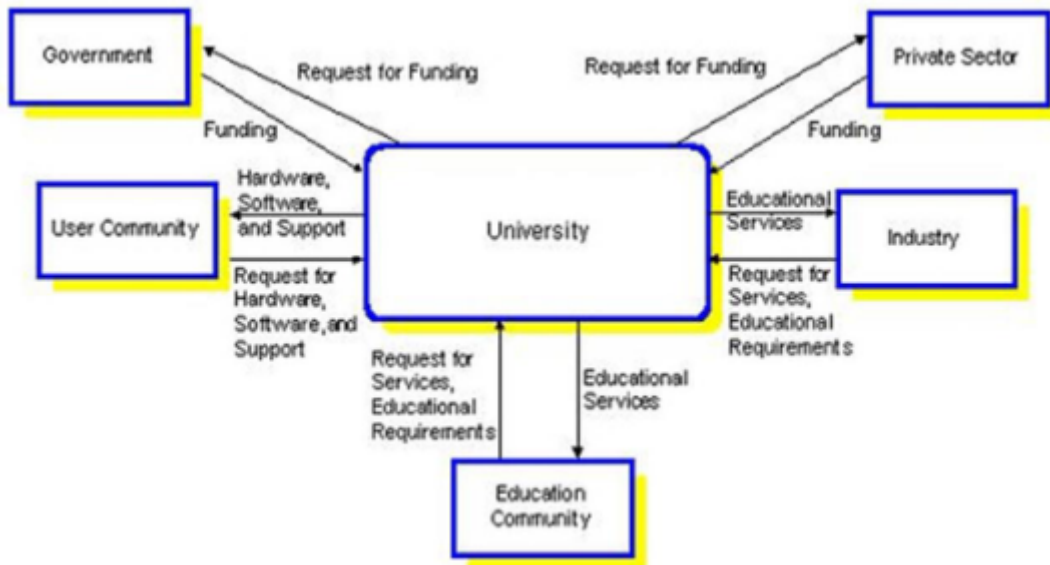
Context diagram

Context diagram = laat zien welke gegevensstromen tussen de buitenwereld en het informatiesysteem bestaan.

- Beschrijft ook de systeemgrenzen: wat het systeem wel of niet moet zijn.
- Zero level data flow diagram.

Type context diagrammen

- **Physical DFD:**
 - Data flow diagram die het model van het huidige systeem vertegenwoordigt.
 - Deze worden getekend wanneer de analist het huidige werksysteem in detail bestudeert.
- **Logical DFD:**
 - Data flow diagram die het model van het voorgestelde systeem vertegenwoordigt.
 - Afkomstig van de fysieke DFD



Context diagram – notatie

- **External entity** = elke entiteit die gegevens levert of informatie ontvangt van het systeem, bijvoorbeeld klant, verkoopafdeling, medewerker.
- **Data flow** = geeft de verplaatsing van gegevens aan, van invoer naar proces of van proces naar uitvoer. Het is gelabeld om aan te geven welke gegevens er stromen, bijvoorbeeld klantgegevens, verkooprapporten.
- **Process** = acties die worden uitgevoerd op invoergegevens om de uitvoergegevens te produceren. Ze krijgen betekenisvolle namen, bijvoorbeeld “rekening voorbereiden”, “omzet berekenen”, “betaling berekenen”.

Symbol	Descriptions
	External Entity
	Data Flow
	Process

Context diagram – regels

- Pijlen mogen elkaar niet kruisen.
- Vierkantjes, cirkels en bestanden moeten namen dragen.
- Ontlede datastromen moeten in evenwicht zijn (alle datastromen in het ontlede diagram moeten de stromen in het originele diagram weerspiegelen).
- **Geen werkwoorden.**
- Geen twee gegevensstromen, vierkanten of cirkels kunnen dezelfde naam hebben.
- Teken alle gegevensstromen rond de buitenkant van het diagram.
- Kies betekenisvolle namen voor datastromen, processen en dataopslag. Gebruik sterke werkwoorden gevolgd door zelfstandige naamwoorden.
- Controle-informatie zoals recordaantallen, wachtwoorden en validatie vereisten zijn niet relevant voor een gegevensstroom diagram.

Context diagram – stappen om te tekenen

- Identificeer externe entiteiten en gegevensstromen van het huidige systeem en teken een fysiek contextdiagram.
- Identificeer gegevensopslag en processen van het systeem en teken eerste niveau fysieke DFD → LATER DFD !!!

- Verken de processen van het eerste niveau en teken DFD van het tweede niveau
→LATER DFD !!!
- Verken de processen van het tweede niveau en teken DFD van het derde niveau
→LATER DFD !!!
- Leid de logische weergave van elke fysieke DFD op de volgende manieren af
 - Verwijder documenten en toon feitelijke gegevens in de gegevensstroom)
 - Verwijder registers en gebruik bestanden als gegevensopslag)
 - Verwijder onnodige processen)
 - Verwijder de gegevensstroom tussen externe entiteiten (indien aanwezig) en toon gegevens doorstroom processen

UML = Unified Modeling Language

UML = Ontwikkeling modelleringstaal voor algemeen gebruik op het gebied van software-engineering, bedoeld om een standaard manier te bieden om het ontwerp van een systeem te visualiseren.

UML voor business modeling

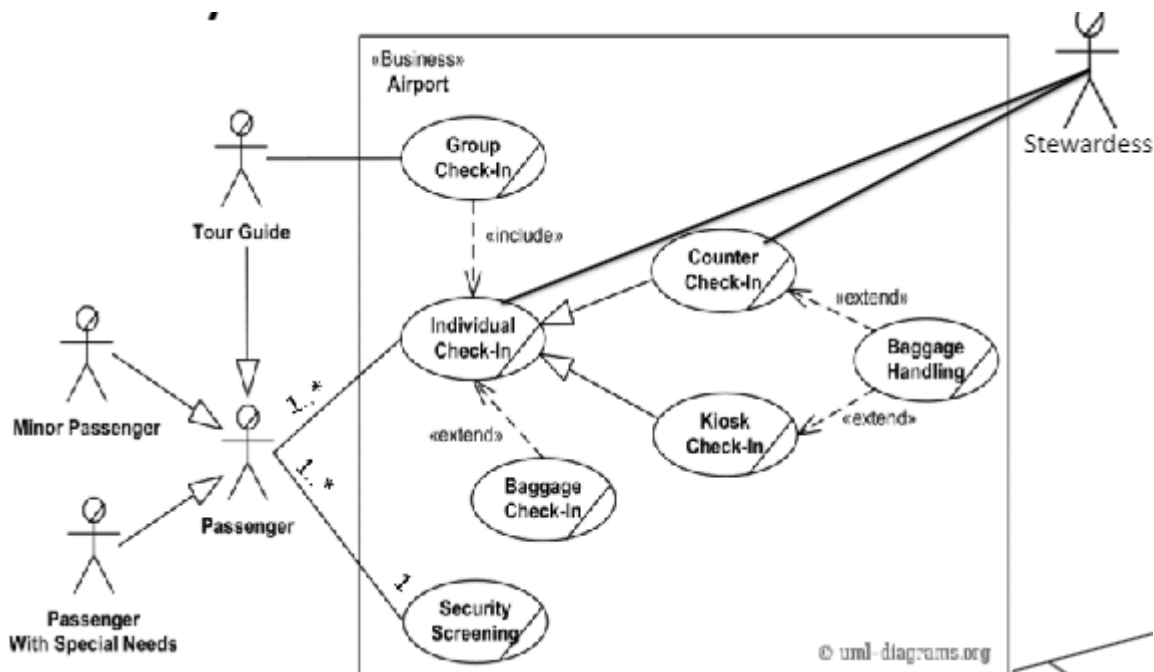
- Doel van UML → ondersteuning voor bedrijfsmodellering
- Hoewel de UML-specificatie geen notatie specifiek voor zakelijke behoeften biedt (zie laatste bolletje in het rood)
- BUCs werden geïntroduceerd in RUP om bedrijfsfunctie, proces of activiteit te vertegenwoordigen die wordt uitgevoerd in het gemodelleerde bedrijf
- Zakelijke actor: rol gespeeld door een persoon of systeem extern aan het gemodelleerde bedrijf en die interageert met het bedrijf
- Een BUC moet een resultaat opleveren van waarneembare waarde voor de zakelijke actor
- Zowel een BUC als een zakelijke acteur zijn niet gedefinieerd in de UML-standaard, dus...
 - Gebruik een UML-tool die deze ondersteunt of...
 - Creëer je eigen zakelijke model stereotypen.

Business Use Case

Waarom Business Use Case

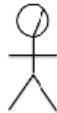
- Voor eenvoudige zakelijke situaties
 - Context diagram kan voldoende zijn om de relevante zakelijke context te modelleren.
 - Wanneer dit niet voldoende is, onderzoekt de analist de zakelijke context verder door gebruik te maken van:
 - Zakelijke use case-model.
 - Zakelijke process model.

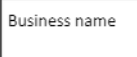
Business Use Case	System Use Case
Een zakelijke use case is een manier waarop een klant of een andere geïnteresseerde partij gebruik kan maken van het bedrijf om het gewenste resultaat te behalen, of het nu gaat om het kopen van een artikel, het verkrijgen van een nieuw rijbewijs, het betalen van een factuur of wat dan ook. Een belangrijk punt is dat een enkele uitvoering van een business use-case alle activiteiten moet omvatten die nodig zijn om te doen wat de klant (of andere actor) wil, en ook alle activiteiten die het bedrijf moet doen voordat het proces vanuit zijn gezichtspunt voltooid is.	Een systeemgebruikscasus is een manier waarop een gebruiker van een computersysteem gebruik kan maken van het systeem om het gewenste resultaat te verkrijgen. Dit zal doorgaans iets zijn waarvan we ons gemakkelijk kunnen voorstellen dat dit in één keer wordt gedaan op een enkele pc of ander apparaat, zoals een geldautomaat of een mobiele telefoon, meestal met een enkele gebruikersinterface, of een klein aantal nauw verwante schermen, zoals een wizard. en het duurt misschien tussen een paar minuten en maximaal een half uur.



Business Use Case – notatie

- **Business use case:** beschrijft één bedrijfsproces als een opeenvolging van (inter)acties tussen een zakelijke actor en een werknemer als geheel om een doel van de zakelijke actor te vervullen. (bijvoorbeeld handmatige verwerking van betalingen, goedkeuring van onkostendeclaraties, beheer van bedrijfsvastgoed).
- **Business actor:** vertegenwoordigt een bedrijfsrol (klant, order intaker,...) die interageert met de bedrijfsomgeving/-proces
- **Business name (business boundary, subject):** een onderwerp van een use case definieert en vertegenwoordigt de grenzen van een bedrijf.
- **Association:** is een relatie tussen classificatie.
- **Generalization:** een generalisatie relatie is een relatie waarin één modelement (het kind) is gebaseerd op een ander modelement (de ouder). Het kind ontvangt alle attributen, bewerkingen en relaties die in het bovenliggende item zijn gedefinieerd.

 Business use case	Describes one business process as a sequence of (inter)actions between a business actor and a worker as a whole to fulfill a goal of the business actor. (e.g., manual payment processing, expense report approval, manage corporate real estate.) The business use case will describe a process that provides value to the business actor , and it describes what the process does . Granularity: document one business use case for every individual business event !!!!!
 Business actor	A business actor represents a business role (customer, order intaker,...) that interacts with the business environment/process.

 Business name	A subject of a use case defines and represents boundaries of a business
 Association	Association is a relationship between classifiers
 Generalization	A generalization relationship is a relationship in which one model element (the child) is based on another model element (the parent). The child receives all of the attributes, operations, and relationships that are defined in the parent.



	<Include>	<Extend>
Use case A	Can not without B	Can exist without B. Does not know that B exists
Use case B	Does not know which use case is calling	Knows to which use case it belongs

Adornment	Semantics
0..1	Zero or 1
1	Exactly 1
0..*	Zero or more
*	Zero or more
1..*	1 or more
1..6	1 to 6
1..3,7..10,15, 19..*	1 to 3 <i>or</i> 7 to 10 <i>or</i> 15 exactly <i>or</i> 19 to many

Business Use Case – stappen om te tekenen

- Identificeer de actoren: iets of iemand dat interageert met het systeem of het gebruikt om een gewenst doel te bereiken.
- Identificeer het doel.
- Definieer de precondities.
- Definieer de postconditions.
- Beschrijf de main flow.
- Beschrijf de uitzonderingen.
- Beschrijf de alternatieve flows.

Problemen en context van requirements gathering

Problemen

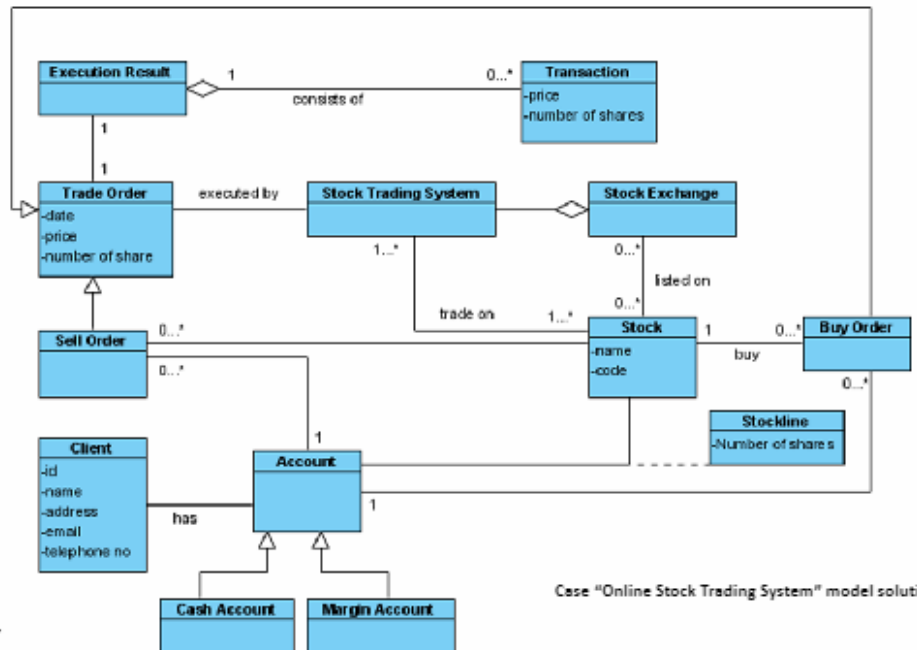
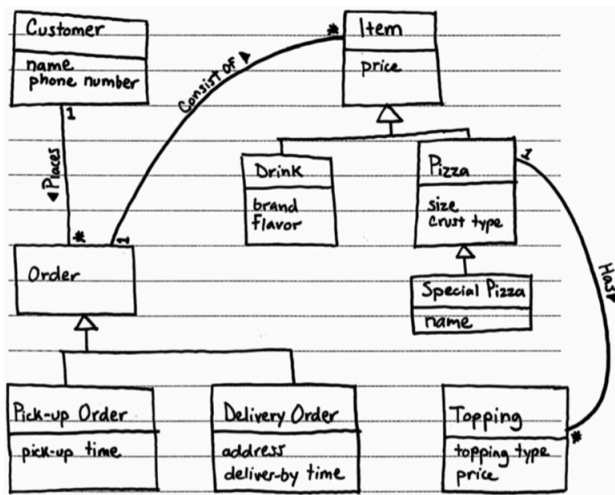
- Onbekend kennisgebied
- Onbekend jargon
- Onbekend probleem
- Onbekend probleemdomein
- Dubbelzinnige communicatie
- Te veel communicatie



Domein model

Domein model = representatie die de essentiële typen objecten in een systeem vastlegt. Het beschrijft 'dingen' in een systeem en hoe deze 'dingen' met elkaar zijn verbonden. Deze 'dingen' kunnen objecten, klassen, interfaces, pakketten of subsystemen zijn die deel uitmaken van het te ontwikkelen systeem.

- Heeft een woordenlijst van termen.
- Domein objecten - domeinklassen.
- Relaties tussen termen.



Eigenschappen en voordelen van een domein model

- Een systeem van abstracties dat geselecteerde aspecten van een kennisgebied, invloedssfeer of activiteit beschrijft.
- Is een visuele representatie van betekenisvolle concepten uit de echte wereld, conceptuele klassen relevant voor het domein dat in software gemodelleerd moet worden.
- Identificeert en relateert belangrijke concepten in een domein.
 - Ook wel "conceptueel modelleren" genoemd.
- Toont associaties en relaties tussen concepten.
 - Bijvoorbeeld, Betaling BETAALT-VOOR Verkoop.
- Toont attributen voor informatieve inhoud.
 - Bijvoorbeeld, Verkoop registreert DATUM en TIJD.
- Concepten van domeinmodel: gegevens die betrokken zijn bij het bedrijf en bedrijfsregels die worden gebruikt met betrekking tot die gegevens.
- Gebruikt om problemen op te lossen die verband houden met dat domein, het bedrijfsleven.
- Gebruikt over het algemeen de woordenschat van het domein, zodat representatie van het model kan worden gebruikt om te communiceren met niet-technische belanghebbenden.

- Omvat geen bewerkingen/functies.
- Beschrijft geen software klassen.
- Beschrijft geen software verantwoordelijkheden.
- Onderdeel van Object-Oriented Analysis.

Klassendiagram = een diagram dat de structuur van het systeem beschrijft door de klassen, de attributen en de bewerkingen van het systeem en de relaties tussen klassen te tonen.

Waarom domeinmodellering?

- Je kent het domein misschien niet goed.
- Je wilt de belangrijkste concepten niet vergeten.
- Je wilt overeenstemming bereiken over een gemeenschappelijke reeks termen.
- Bereid je voor op ontwerp.

Domein model – stappen om te tekenen

- Formuleer een probleemstelling voor het te ontwikkelen systeem: Definieer het probleem dat het systeem moet oplossen en bepaal de scope van het domein waarin het systeem opereert.
- Identificeer concepten (dit zijn de klassen en objecten): Identificeer de belangrijkste concepten, entiteiten, en objecten die relevant zijn voor het probleemdomein.
- Ontwikkel een gemeenschappelijke woordenschat, woordenboek, glossarium: Creëer een lijst met termen en hun betekenissen om een gemeenschappelijk begrip te bevorderen.
 - Tel de voorkomens en maak een alfabetische lijst.
 - Creëer een eerste diagram van domeinklassen.
- Identificeer associaties tussen concepten, inclusief de leesrichting: Definieer de relaties en associaties tussen de verschillende concepten. Markeer ook de richting van deze associaties.
- Wijs attributen toe aan de concepten: Bepaal de eigenschappen en kenmerken (attributen) van elke klasse. Bijvoorbeeld, een Persoon klasse kan attributen hebben zoals naam, leeftijd, enz.
- Controleer op multiplicaties en geef deze aan in het domeinmodel: Identificeer eventuele veel-op-veel-relaties tussen klassen en markeer deze in het model. Dit kan bijvoorbeeld aangeven dat één klasse aan meerdere instanties van een andere klasse is gekoppeld.
- Itereer en verfijn het model: Herzie en verbeter het model op basis van feedback en voortschrijdend inzicht. Voeg meer details toe en zorg voor een nauwkeurige weergave van het domein.

“Microwave oven: how to heat food?”

ID	BUC_08, version v3.0	
Title	The heating of food	
Actor(s)	Cook	
Precondition(s)	The microwave oven is inactive, but it is plugged in The door is closed The food to heat is in close reach	
Normal flow	Step	Description
	01	The user opens the door
	02	The light goes on
	03	The user puts the food into the oven
	04	The user closes the door
	05	The light turns out
	06	The user presses the button once to set the working time to 60 seconds

“Microwave oven: how to heat food?”

Normal flow	Step	Description
(continued)	07	The light goes on
	08	Microwave tube starts
	09	The working time becomes visible on the display
	10	The working time decreases, and it is visible in the window. When the time is up:
	11	The light turns out
	12	A beep sounds
	13	The user opens the door
	14	The light goes on
	15	The user takes out the food
	16	The user closes the door
	17	The light turns out
Post condition	The food is heated	

“Microwave oven: how to heat food?”

ID	BUC_08, version v3.0	
Title	The heating of food	
Actor(s)	Cook, the user	
Precondition(s)	The microwave oven is inactive, but it is plugged in The door is closed The food to heat is in close reach	
Normal flow	Step	Description
	01	The <u>user</u> opens the <u>door</u>
	02	The <u>light</u> goes on
	03	The <u>user</u> puts the <u>food</u> into the <u>oven</u>
	04	The <u>user</u> closes the <u>door</u>
	05	The <u>light</u> turns out
	06	The <u>user</u> presses the <u>button</u> once to set the <u>working time</u> to 60 <u>seconds</u>

“Microwave oven: how to heat food?”

Normal flow	Step	Description
(continued)	07	The <u>light</u> goes on
	08	<u>Microwave tube</u> starts
	09	The <u>working time</u> becomes visible on the <u>display</u>
	10	The <u>working time</u> decreases, and it is visible in the <u>window</u> . When the <u>time</u> is up:
	11	The <u>light</u> turns out
	12	A <u>beep</u> sounds
	13	The <u>user</u> opens the <u>door</u>
	14	The <u>light</u> goes on
	15	The <u>user</u> takes out the <u>food</u>
	16	The <u>user</u> closes the <u>door</u>
	17	The <u>light</u> turns out
Post condition	The food is heated	

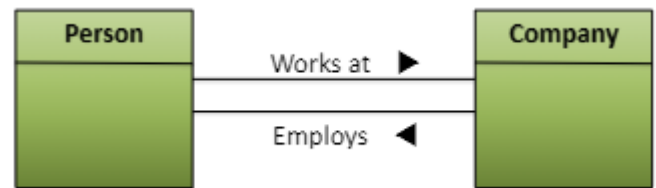
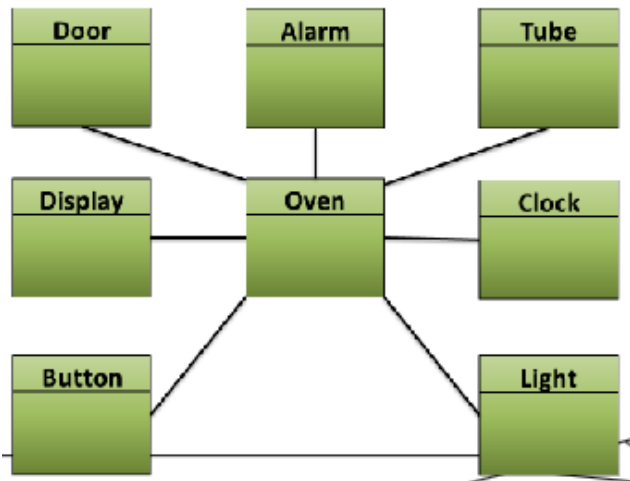
Nouns (1)	#	Nouns (2)	#	Nouns (3)	#
Door	4	Button	1	Display	1
Light	6	Working time	3	Window	1
Food	2	Seconds	1	Time	1
Oven	1	Microwave tube	1	Beep	1

Nouns (1)	#	Nouns (2)	#	Nouns (3)	#
Beep	1	Food	2	Seconds	1
Button	1	Light	6	Time	1
Display	1	Microwave tube	1	Window	1
Door	4	Oven	1	Working time	3

Nouns	#	Definitions and comments	Nouns	#	Definitions and comments
Button	1	Tangible thing. Button is a critical concept.	Beep	1	Concept. Beep is a critical concept.
Display	1	Tangible thing. Display is a critical concept.	Button	1	Tangible thing. Button is a critical concept.
Door	4	Tangible thing. Door is a critical concept.	Display	1	Tangible thing. Display is a critical concept.
Food	2	Interaction. Food is not part of the system, the oven ...	Door	4	Tangible thing. Door is a critical concept.
Light	6	Tangible thing. Light is a critical concept.	Food	2	Interaction. Food is not part of the system, the oven ...
Microwave tube	1	Tangible thing. Microwave tube is a critical concept.	Light	6	Tangible thing. Light is a critical concept.
Oven	1	Tangible thing. Oven tube is a critical concept. It is the "case" that keeps the whole together.	Microwave tube	1	Tangible thing. Microwave tube is a critical concept.
Seconds	1	Attribute. Unit of measure for working time.	Oven	1	Tangible thing. Oven tube is a critical concept. It is the "case" that keeps the whole together.
Time	1	Concept. Time means the same as working time.	Seconds	1	Attribute. Unit of measure for working time.
Window	1	Tangible thing. Window means the same as a display.	Time	1	Concept. Time means the same as working time.
Working time	3	Concept. Working time is a critical concept. We need a clock, a timepiece	Window	1	Tangible thing. Window means the same as a display.
			Working time (= clock)	3	Concept. Working time is a critical concept. We need a clock, a timepiece

Domain Class	Nouns	#	Definitions and comments
Alarm	Beep	1	Concept. Beep is a critical concept.
Button	Button	1	Tangible thing. Button is a critical concept.
Clock	Working time (= clock)	3	Concept. Working time is a critical concept. We need a clock, a timepiece
Display	Display	1	Tangible thing. Display is a critical concept.
Door	Door	4	Tangible thing. Door is a critical concept.
Light	Light	6	Tangible thing. Light is a critical concept.
Oven	Oven	1	Tangible thing. Oven tube is a critical concept. It is the "case" that keeps the whole together.
Tube	Microwave tube	1	Tangible thing. Microwave tube is a critical concept.

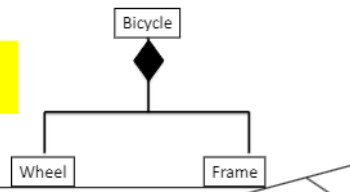
Nouns	Explanation - description
Alarm	A short, typically high-pitched sound or tone, produced by the oven. It serves the purpose to notify when the timer is ended.
Button	The control element that allow you to operate the microwave oven and set parameters for cooking or heating food.
Clock	The clock on a microwave oven is a digital or analog timekeeping feature that displays the current time. It serves as a standard clock, similar to those found on alarm clocks or kitchen wall clocks, and has several functions and purposes like time display and timer.
Display	A digital or sometimes analog screen that provides information about the microwave's settings, cooking time, and other important details.
Door	A crucial safety feature that serves several purposes. It is a hinged or sliding panel on the front of the microwave oven that opens and closes to access the microwave's cooking chamber
Light	An interior light designed to illuminate the cooking chamber when the microwave is in operation.
Oven	The crucial kitchen appliance that uses electromagnetic radiation in the microwave frequency range to cook, heat, and reheat food quickly and efficiently. Microwave ovens are a common and convenient way to prepare meals, snacks, and beverages.
Tube	A high-powered vacuum tube that generates the microwave radiation used for cooking or heating food in a microwave oven.



– Composition

- A part can always only belong to a single entity
- The part of the object can not independently continue to exist if the object itself, where the part is part of, disappears (not shared association)
- Remark: use of a composite relationship is preferable to the aggregation relationship

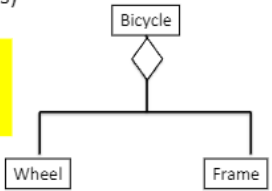
The bicycle is one whole
The bicycle has a composition relationship with its parts



– Aggregation

- Indicates that one or more domain classes are part of another, different class
- A shared association (also part of others)

For a cyclist the bicycle is NOT one whole.
The weather for example will determine the choice of the wheels which are put under the frame.



Multiplicity	
1	Exactly 1
2	Exactly 2
1 .. 5	From 1 to 5 (included)
6, 7	6 or 7
1 .. *	Undefined number but at least 1
0 .. *	Undefined number, eventually 0 Is the same as *
*	Undefined number, eventually 0 Is the same as *
0 .. 1	0 or 1 Is the same as 0, 1

Modellen & modellen gebruiken

Model = abstractie van de bestaande werkelijkheid.

- Representatie: brengt de werkelijkheid in kaart.
- Reductie: reduceer de weergegeven werkelijkheid.
- Pragmatisch: geconstrueerd voor een speciaal doel.

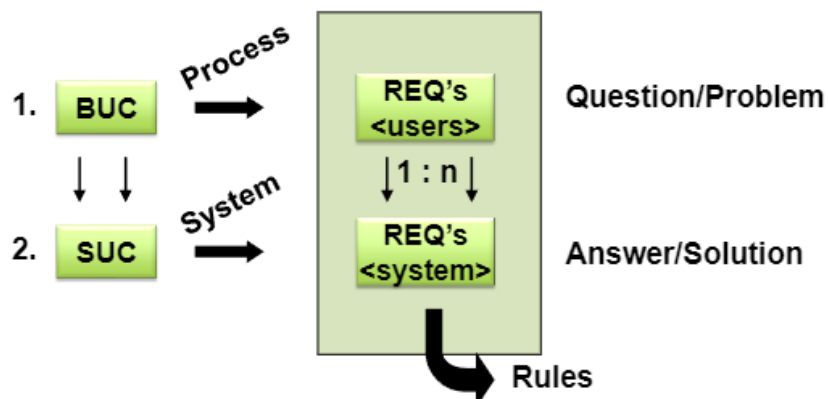
Modellen gebruiken: a combination of natural language and requirements models.

- Informatie in afbeeldingen is sneller te begrijpen en te onthouden.
- Maakt gerichte modellering van één perspectief mogelijk.
- Passende abstracties van de werkelijkheid kunnen worden gespecificeerd door de modelleringstaal voor het specifieke doel te definiëren.

Meest gebruikte modellen

- **System Use Case Diagram (+ Description):** Represents a user's interaction with the system.
- Entity Relationship Diagram: Describes the data or information aspects of a business domain or its process requirements.
- Class Diagram (OO): Describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects
- Data Flow Diagram: Shows the detailed functionality.
- **Activity diagram:** Shows the overall flow of control.
- **State chart:** Shows the behavior.
- *Sequence Diagram:* Shows how processes operate with one another and in what order.

Goal model = Een doel is een doelbewuste beschrijving van één of meer belanghebbenden over een gewenste karakteristieke eigenschap van het systeem. Vaak in natura language, maar ook in termen van modellen en/of bomen.



System Use Case

System Use Case Models + Description = use cases helpen bij het onderzoeken en documenteren van de functionaliteit vanuit het perspectief van gebruikers van het systeem.

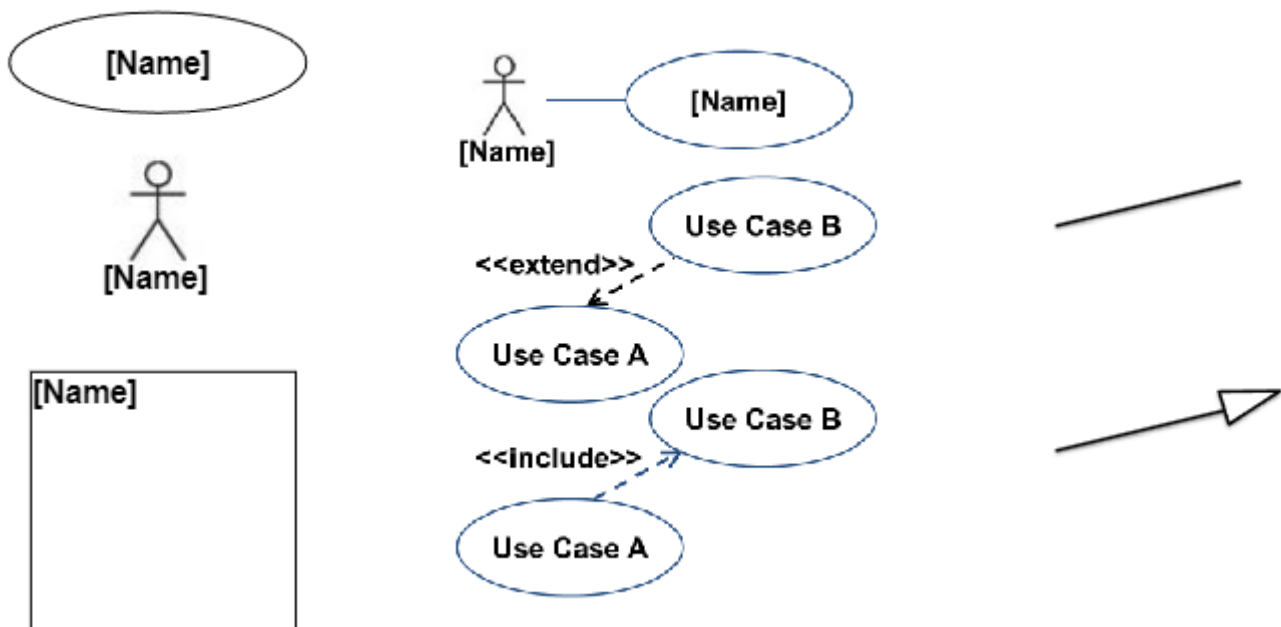
- System use case diagrams.
- System use case specifications (= descriptions)

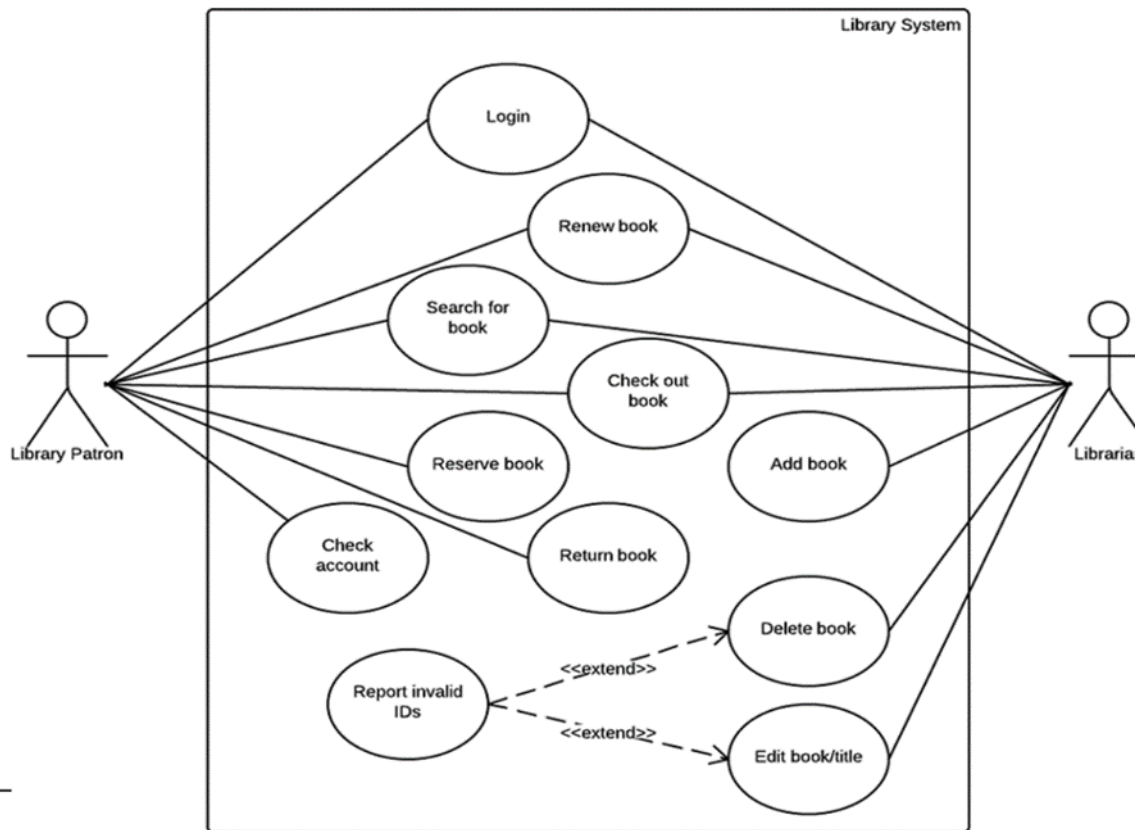
System Use Case Diagrams

- Vanuit een extern gezichtspunt.
- Het tonen van de relatie met de context, en (sommige) relaties tussen de functionaliteiten.
- Elementen:
 - Use cases.
 - Actoren in de context.
 - Systeemgrens.
 - Relaties tussen de elementen.

System Use Case – notatie

- **Use case:** werkwoorden.
- **Actor:** personen en/of andere systemen die met het systeem interageren.
- **System boundary:** bepaalt wat zich binnen (de use cases) en buiten (personen en andere systemen) bevindt.
- **Communication:** tussen acteur en gebruiksscenario.
- **Extend:** geeft aan dat het gedrag van de use-case van de extensie onder bepaalde omstandigheden in de uitgebreide use-case kan worden ingevoegd.
- **Include:** geeft aan dat de use case waarnaar de pijl wijst, is opgenomen in de use case aan de andere kant van de pijl.
- **Association:** relatie tussen classificaties.
- **Generalization:** relatie waarbij één modelement (het kind) gebaseerd is op een ander modelement (de ouder). Het onderliggende item ontvangt alle attributen, bewerkingen en relaties die in het bovenliggende item zijn gedefinieerd.





Hoe kan je use cases vinden?

- Vind eerst actoren.
- Identificeer de grenzen van het systeem.
- Definieer use cases voor elke actor.
- Definieer voor elke use case:
 - Randvoorwaarden.
 - De interactie.
 - De mogelijke uitzonderingen.
- Beschrijf elke use case en teken het use case diagram.
- Kies de systeemgrens:
 - Wat bouw je?
 - Wie gaat het systeem gebruiken?
 - Wat wordt er nog meer gebruikt dat je niet bouwt
- Vind primaire actoren en hun doelen:
 - Brainstorm eerst over de primaire actoren,
 - Wie start en stopt het systeem?
 - Wie wordt op de hoogte gesteld als er fouten of mislukkingen zijn?
- Definieer gebruiksscenario's die aan de gebruiksdoelen voldoen:
 - Bereid een lijst met actoren en doelen op (en niet de actor -takenlijst).
 - Over het algemeen één gebruiksscenario voor elk gebruiksdoel.
 - Noem het gebruiksscenario vergelijkbaar met het doel van de gebruiker.

Functional requirements models

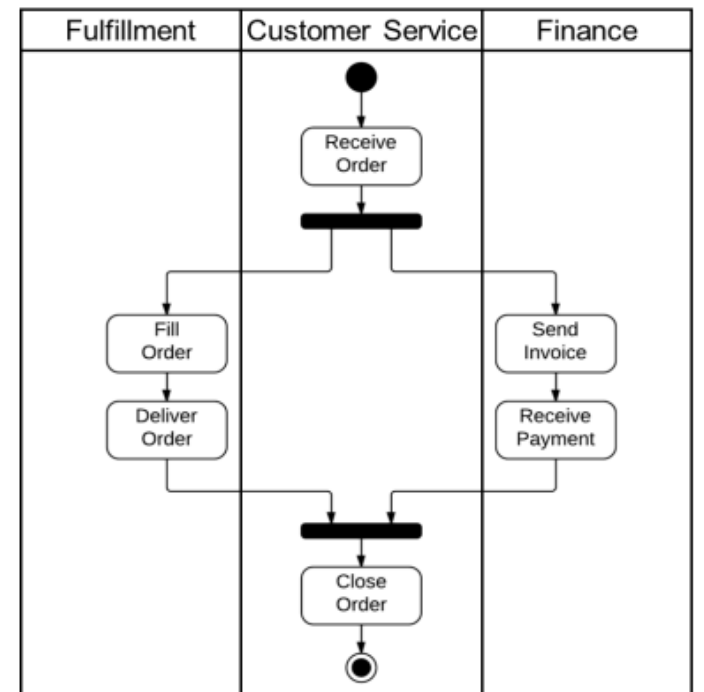
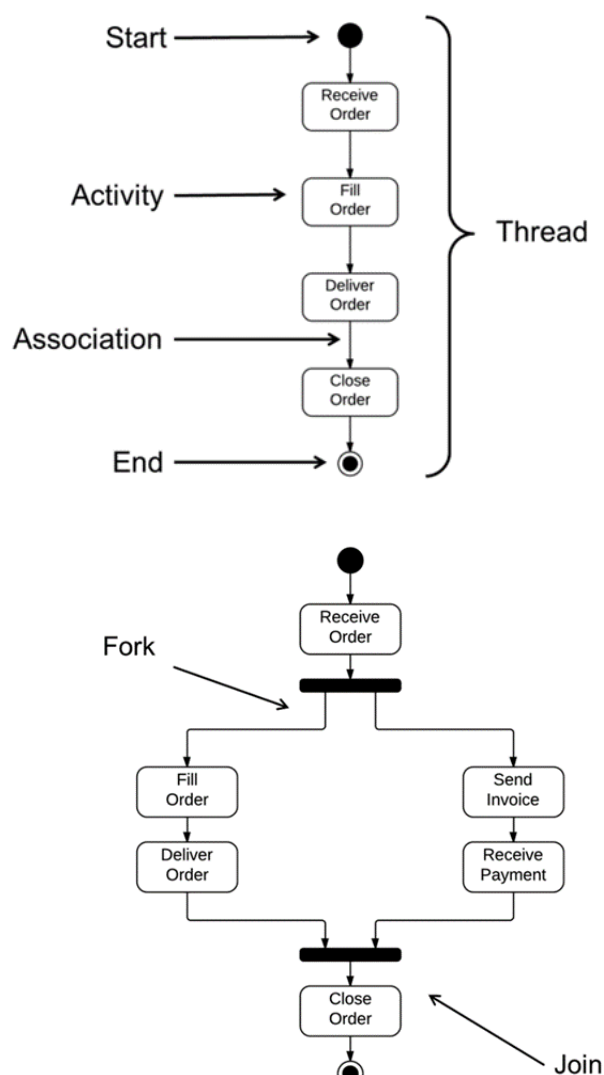
Models

- De nadruk ligt op het verwerken van invoergegevens uit de omgeving van het systeem om uitvoergegevens van het systeem te creëren.
- Meest abstracte vorm: System Use Case Model.
- Andere:
 - Data flow diagram.
 - Activiteitendiagram (UML).

Activity diagram

Activity diagram = is een type diagram binnen de Unified Modeling Language (UML) dat wordt gebruikt om de stroom van activiteiten in een systeem of proces te modelleren.

- Focus op de activiteiten van een proces en de controle ertussen.
- Beschrijft de volgorde van activiteiten.
- Procedure die voortvloeit uit individuele acties.
- Draden kunnen parallel of voorwaardelijk zijn.
- Vaak gebruikt om de scenario's (normale, alternatieve en uitzondering stromen) van de use case te verduidelijken/detailleren.



Activity diagram – stappen om te tekenen

- Stroomschema bestaande uit activiteiten uitgevoerd door het systeem
- Activity diagram niet precies stroomschema: extra mogelijkheden: vertakking, parallelle stroming, zwembaan, etc. Voordat u het activiteitendiagram tekent
 - Duidelijk begrip van de gebruikte elementen
 - Het hoofdelement = activiteit zelf
 - Activiteit = functie uitgevoerd door systeem
- Na het identificeren van activiteiten
 - Begrijp hoe activiteiten verband houden met beperkingen en omstandigheden
- Identificeer dus de volgende elementen voordat u een activiteitendiagram tekent:
 - Activiteiten
 - Associatie
 - Voorwaarden
 - Beperkingen
- Teken vervolgens het diagram

Voordelen activity diagram	Nadelen activity diagram
Makkelijk te leren	Valkuil: te veel details specificeren
Breed toepasbaar	50% van de activiteiten zou (bijna) voldoende zijn
Kan complexe, tijdgerelateerde procedures tonen	Weinig praktische informatie in UML-handleiding

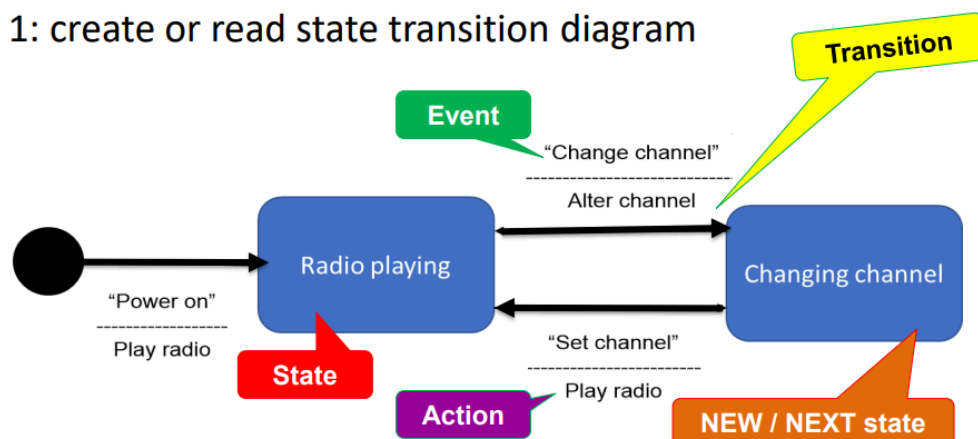
Behavioral requirements models

Models

- Voor het modelleren van het dynamische gedrag van een systeem
- De nadruk ligt op systeem toestanden en gebeurtenissen die die toestand kunnen veranderen.
- Gedragsmodellen.
- UML-status diagrammen:
- Gebaseerd op de principes van eindige toestandsmachines.
 - Modelleren elementen: toestand, begin- en eindtoestand, toestand transitie, gelijktijdigheid.

State Transition Diagram

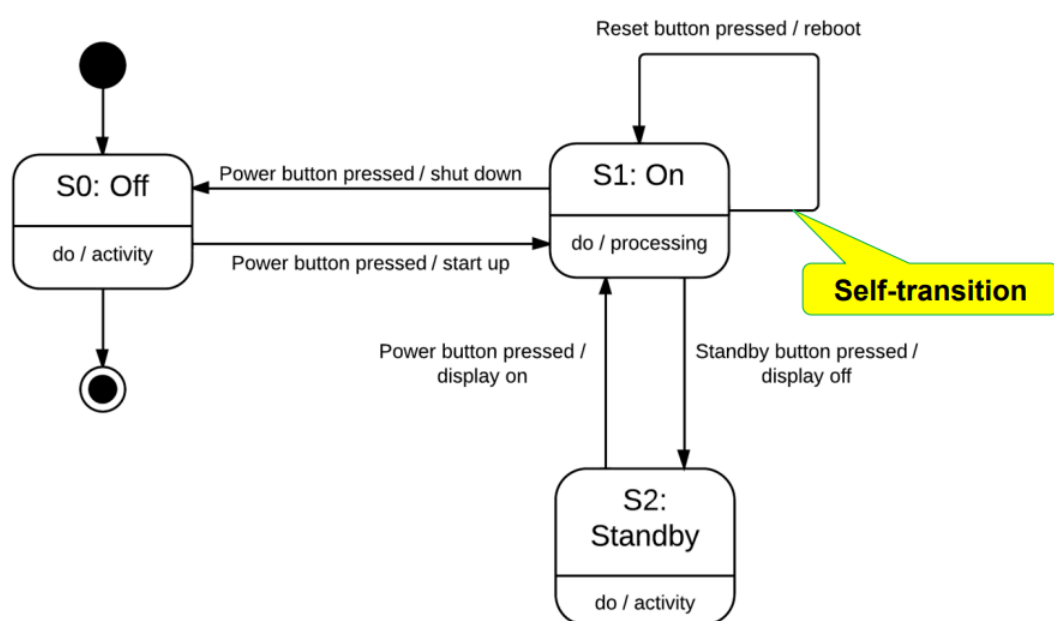
- Step 1: create or read state transition diagram



- Step 2: create state transition table

EVENT	Power on	Change channel	Set channel
STATE			
NULL	A: Play radio S _{new} : Radio playing	n.a.	n.a.
Radio playing	n.a.	A: Alter channel S _{new} : Channel changing	n.a.
Channel changing	n.a.	n.a.	A: Play radio S _{new} : Radio playing

Callouts: 'Events' points to the header row. 'States' points to the first column. 'Action' points to the actions in the table. 'NEW / NEXT state' points to the S_{new} values in the table.



State Transition Diagram

- Beschrijft het gedrag van een systeem
- Alle mogelijke toestanden waarin een systeem kan verkeren
- Toestandsveranderingen als gevolg van gebeurtenissen
- Acties zijn korte effecten van transities

State Transition Diagram – stappen om te tekenen

- Definieer staten.
- Beschrijf toestanden (zodat anderen het kunnen begrijpen).
- Teken overgangen.
- Definieer gebeurtenissen / bewaking voorwaarden.
- Acties definiëren.
- Maak een status overgangstabel.

Voordelen state transition diagram	Nadelen state transition diagram
Geschikt voor het modelleren van interfaces	Kan voor buitenstaanders moeilijk te lezen zijn
Geschikt voor simulatie en testen	

Natural Language

Natural language = elke stakeholder moet het makkelijk kunnen verstaan, het is universeel.

- The chicken is ready to eat.
- They are hunting dogs.
- Ik leer kinderen tekenen.

Taal effecten = taal is subjectief.

- Wat wordt er gezegd versus wat wordt er bedoeld.
- Requirements moeten duidelijk zijn en niet te veel interpretatie hebben.

Vijf meest relevante taal transformaties

- Nominalization: het gebruik van werkwoorden voor complexe processen te.
- Nouns without reference index: niet precies.
- Universal quantifies: “never”, “always”, “none”.
- Incompletely specified conditions: “in case”, “when”, “dependent on”.
- Incompletely specified process words: “to transfer”, van waar naar waar?

Dealing with transformations

Five steps to write requirements according to the template

- Determine legal obligation/relevance.
- Determine the core of the requirement: the required functionality.

- Characterize the activity: the way the system works:
 - Autonomous system activity: the system carries out the process by itself,
 - User interaction: the system provides the user with process functionality.
 - Interface requirement: the system carries out the process dependent on a third factor (for instance a different system), remains passive and waits for an external event.
- Insert objects: further elements are required for the completion.
- Determine conditions

Redenen voor documentatie

- Eisen 'kunnen' een lange levensduur hebben.
- Bereikbaarheid voor veel personen.
- Juridische relevantie / compliance.
- We hebben een goed overzicht nodig.
- Eisen kan zeer complex worden.
- Behoeften van belanghebbenden.

Twee aspecten

- Moet leesbaar zijn: De structuur van het document, hoe het is georganiseerd en hoe de stroom wordt ondersteund de lezer om individuele vereisten in context te plaatsen.
- Moet beheersbaar zijn: Kwaliteiten van individuele eisen, duidelijkheid en nauwkeurigheid en hoe ze zijn onderverdeeld in afzonderlijke traceerbare items.

Perspectives on system to be developed

- **Data perspective:** structuur van data.
- **Functional perspective:** Welke informatie (data) uit de systeemcontext wordt door het systeem gemanipuleerd en welke gegevens van het systeem naar de systeemcontext stromen.
- **Behavioral perspective:** Het documenteren van het systeem, ingebed in de systeemcontext, in een staat georiënteerde manier.

Three main sections

- Introduction
- Overall description
 - Constraints
- Specific requirements
 - Functional requirements (grouped)
 - Quality requirements



Waarom hebben wij een glossary nodig?

- Homoniemen.
- Synoniemen.
- Gangbare termen met een specifieke betekenis in de context van het project.
- Specifieke voorwaarden van het domein.
- Technische termen.
- Acroniemen en afkortingen.
- Bedrijfsspecifiek en aanvullend projectspecifiek.

Requirements schrijven

- Functional requirements:
 - Beschrijf een mogelijkheid die de stakeholder nodig heeft of een actie die het product moet ondernemen: "The product shall be able to do a specific thing (for a specific actor)".
 - Een enkele zin met een enkel indicatief werkwoord - gebruik eenvoudige directe taal.
 - Geen oplossing schrijven.

Guidelines

- Korte en bondige zinnen en paragrafen.
- Eén eis per zin (geen samengestelde eisen), geen nesting.
- Consistente terminologie.
- Vermijd generalisaties, duidelijke verwijzingen.
- Gebruik zorgvuldig 'moeten', 'kan' en soortgelijke woorden ('zullen' is beter).

Quality gateway = elke individuele eis moet de kwaliteit gateway passeren om aan de specificatie te worden toegevoegd.

Fit criteria

Fit criteria = 'Fit' betekent dat een oplossing volledig voldoet aan het gestelde vereiste.

- Het is noodzakelijk om een kwantificering aan de vereiste te koppelen.
 - Maak het meetbaar.
 - Dit is de uitkomst.
- Het geschiktheids criterium kan het gedrag, de prestatie of iets dergelijks kwantificeren:
 - Dus belangrijker voor kwaliteitseisen.
 - Functionele eisen moeten al toetsbaar zijn.
- Fit criteria zijn van toepassing op zowel functionele als niet-functionele eisen.
- Maak het mogelijk om vast te stellen of aan een eis is voldaan of niet.
- Meestal is het nodig om met de belanghebbende over geschiktheids criteria te onderhandelen.

Fit criteria – functional user story: "As a registered user, I want to be able to customize my profile by uploading a profile picture so that my profile reflects my identity."

Acceptance Criteria:

1. The system should provide an option for users to upload a profile picture
2. The uploaded picture should meet the specified file format (e.g., JPEG, PNG)
3. The system should display a preview of the selected picture before confirmation
4. Upon confirmation, the uploaded picture should be saved to the user's profile
5. Users should be able to update or remove their profile picture at any time.

Fit criteria – non-functional user story: “As a system administrator, I want the data retrieval process to be swift and efficient (1000 records from the database in less than 3 seconds), so that I can quickly generate reports and provide timely information to end-users”

Acceptance Criteria:

1. The system should retrieve and display 1000 records from the database in less than 3 seconds.
2. The system's response time for data retrieval shall not significantly increase, maintaining a maximum increase of 20% as the database grows.
3. The system's response time shall meet specified benchmarks for consistent performance based on industry standards.

Fit criteria – functional system requirement: “When an instructor wants to add content, the system must give the instructor the ability to upload various types of educational content, including Word documents, Excel sheets, PowerPoint presentations, MP4 videos, and interactive quizzes”

Acceptance Criteria:

1. The system must successfully accept and process uploads of Word documents (.docx), Excel sheets (.xlsx), PowerPoint presentations (.pptx), MP4 videos, and interactive quiz files, without data corruption.
2. Instructors should be able to upload Word documents, Excel sheets, and PowerPoint presentations up to 50 MB in size, and MP4 videos and interactive quizzes up to 500 MB, with the system preventing uploads exceeding these limits.

Fit criteria – non-functional system requirement: “When an instructor wants to add content, the system should facilitate uploads from various devices and operating systems commonly used by instructors, such as Windows, macOS, and common web browsers (Chrome, Firefox, Safari).”

Acceptance Criteria:

1. The system must support file uploads from Windows 10 (or the latest version) and macOS 10.14 (or the latest version), ensuring compatibility with the two major operating systems used by instructors.
2. File uploads should be fully functional on the latest versions of commonly used web browsers, including Google Chrome, Mozilla Firefox, and Apple Safari.
3. The system shall support Microsoft Edge for users on Windows and Safari for users on macOS.

Volère card

Volere kaart = naar een hulpmiddel dat wordt gebruikt in requirements engineering, specifiek in het kader van de Volere-methode.

- De Volere-methode is een aanpak voor het vastleggen, documenteren en beheren van systeemvereisten gedurende de levenscyclus van een project.

Requirement #	Unique number of the requirement	Priority	Based on the words Shall/Must (=1), Should (=2), Will (=3)
Requirement title	Kind of summary indicating the requirement		
Requirement Type	PERFUMeS or business requirement	Use Case	Number of the linked use case in use case diagram and description
Description	Give a SMART description in natural language of the requirement. Describe the intend of the requirement. Use the BLUEPRINTS of a user story and a system requirement		
Rationale	The reason WHY this requirement needs to be developed		
Source	Stakeholder (group) of the requirement. For which user this requirement is defined?		
Fit Criterion :	Acceptance criterion defined in a quantified manner. To be used for acceptance testing		
Supporting material	Link to further material	Annotation¹	Eventual remarks concerning the requirement

Requirements elicitation

Wat is **requirements elicitation** = is een actieve inspanning om informatie te verkrijgen van belanghebbenden en experts op een bepaald gebied.

- To elicit(uitlokken):
 - Naar voren halen of naar buiten brengen.
 - Roepen of uittrekken.

Requirements Gathering



- Like collecting seashells
- Take what you see
- More reactive, less proactive

versus

Requirements Elicitation



- Like archeology
- Planned, deliberate search
- More proactive, less reactive

Elicitation problems

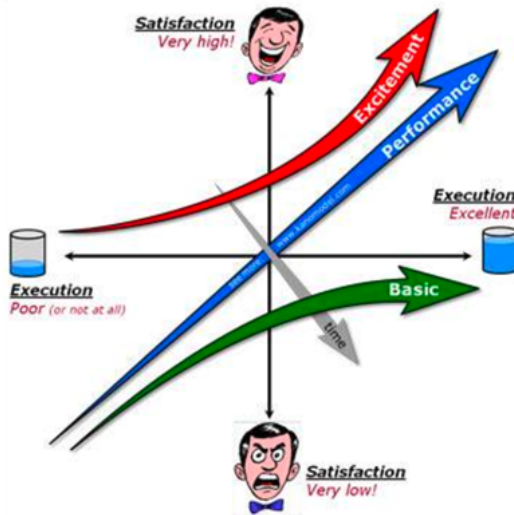
- Meer sociaal dan technologisch.
- De aard van de problemen zijn meer psychologisch en sociaal dan technisch.
- Articulatieproblemen:
 - Moeilijkheid om behoeften duidelijk uit te drukken
 - Niet bewust zijn van eigen behoeften
 - Niet begrijpen hoe technologie kan helpen
 - Bang zijn om incompetent over te komen vanwege technologische onwetendheid
 - Geen beslissingen nemen omdat men niet in staat is om zowel de gevolgen te voorzien, als de alternatieven te begrijpen, als een globaal beeld te hebben
 - Niet goed naar klanten luisteren.
- Communicatie problemen:
 - Andere cultuur en woordenschat.
 - Verschillende belangen op het te ontwikkelen systeem.
 - Ongepaste communicatiemedia (d.w.z. diagrammen die klanten en gebruikers niet begrijpen).
 - Persoonlijke of politieke conflicten.
- Cognitieve limitaties
 - Het probleemdomein niet begrijpen
 - Het doen van aannames over het probleemdomein
 - Het doen van aannames over technologische aspecten
 - Overmatige vereenvoudigingen
- Menselijk gedrag
 - Het probleemdomein niet begrijpen
 - Het doen van aannames over het probleemdomein
 - Het doen van aannames over technologische aspecten
 - Overmatige vereenvoudigingen
- Technische problemen
 - Complexiteit van probleemdomeinen
 - Complexiteit van vereisten
 - Veranderende eisen, hoe meer je ziet, hoe meer je nodig hebt
 - Wijzigingen in hardware en software
 - Meerdere vereisten bronnen
 - Onduidelijke informatiebronnen

Driven by

- System context
 - Context diagram.
 - BUC model & description.
 - Domain model.
- Verschillende bronnen voor requirements:
 - Stakeholders.
 - Bestaande documenten.
 - Legale systemen.

Kano model

Kano model = proactief drie soorten vereisten te helpen ontdekken, classificeren en integreren die de klanttevredenheid beïnvloeden.



Functional form of the question	
If you have to wait for less than 2 minutes before the customer service rep answers your call, how do you feel?	1. I like it that way 2. It must be that way 3. I am neutral 4. I can live with it that way 5. I dislike it that way
If you have to wait for more than 2 minutes before the customer service rep answers your call, how do you feel?	1. I like it that way 2. It must be that way 3. I am neutral 4. I can live with it that way 5. I dislike it that way
Dysfunctional form of the question	

www.xserve.in

contact@xserve.in



Category (Attribute Perception)	When attribute is present	When attribute is absent
One-dimensional (O)	Satisfied	Dissatisfied
Must be (M)	No feeling	Dissatisfied
Attractive (A)	Satisfied	No feeling
Indifferent (I)	No feeling	No feeling
Reverse (R)	Dissatisfied	Satisfied
Questionable (Q)	Normally answers do not fall into this category. Q signifies that either the question was phrased incorrectly or the customer misunderstood the question. Customer may have selected a wrong answer by mistake	

Feature ↕	Attractive	Irrelevant	Must be	Performance	Questionable	Reverse	Category ↕
#1 Making phone calls	3.4% (34)	9.8% (99)	62.7% (635)	20.4% (206)	2.9% (29)	0.9% (9)	Must be*
#2 High resolution of ...	20.3% (205)	14.8% (150)	24.5% (248)	36.2% (366)	2.7% (27)	1.6% (16)	Performance*
#3 Free noise-cancel...	54.4% (551)	23.4% (237)	4% (40)	13.9% (141)	3.1% (31)	1.2% (12)	Attractive*
#4 Different ringtones	19.2% (194)	58.7% (594)	9.3% (94)	9.4% (95)	2.2% (22)	1.3% (13)	Irrelevant*
#5 Long battery life	7.5% (76)	8.7% (88)	33.4% (338)	46.3% (469)	2.6% (26)	1.5% (15)	Performance*
#6 Contactless paym...	21.2% (215)	44.4% (449)	16.6% (168)	11.1% (112)	2.8% (28)	4% (40)	Irrelevant*
#7 Storage capacity	12.7% (129)	11.7% (118)	31% (314)	40.6% (411)	1.7% (17)	2.3% (23)	Performance*
#8 Permanent tracking	4.4% (45)	33.6% (340)	2.1% (21)	2.5% (25)	3.8% (38)	53.7% (543)	Reverse*
#9 Free case	57.2% (579)	29.1% (294)	1.5% (15)	8.1% (82)	2.9% (29)	1.3% (13)	Attractive*
#10 Waterproof	42.2% (427)	21.4% (217)	11.5% (116)	21.3% (216)	2.4% (24)	1.2% (12)	Attractive*

Elicitation process



Elicitation technieken

Survey techniques

- Interviews
- Survey/questionnaires

Creativity techniques

- (Paradox) brainstorming
- Change of perspective
- Analogies
- Role playing

Observation techniques

- Apprenticing
- Field observation

Document-centric techniques

- System archaeology
- Perspective based reading
- Reuse of requirements
- Existing document analysis
- Interface analysis

Supporting techniques

- Mind mapping
- Workshops
- Prototyping
- Focus groups
- Process modeling